SCE 672: Advanced Robotics

Term Project Assessment 1: System Provisioning, Keyboard Teleoperation, Telemetry, and ROS 2 Integration

Student: Ameen Ur Rehman (202527430) **Marks:** 10
Instructor: Prof. Aamer Iqbal Bhatti
Institution: King Fahd University of Petroleum & Minerals
Code and data: `https://github.com/AmeenUrRehman/RoboMaster-Advanced-Robotics` (private repository; folder `Assignment 1`)

Abstract

This report documents the provisioning of a bare-metal Ubuntu 24.04 / ROS 2 Jazzy environment for a DJI RoboMaster EP Core, a standalone keyboard teleoperation and telemetry script written against the native Python SDK, and a ROS 2 integration over the robot's own Wi-Fi access point. All numbers in the report come from logs recorded on the robot during the project; every terminal figure is an image rendered from the *captured text* of the real command (the text files are in `logs/captures/` and `logs/`), and nothing was typed in by hand. The main findings are: (i) a virtual environment with system site-packages, a pinned `numpy<2`, and a shell loader that sets `PYTHONNOUSERSITE` keeps NumPy 1.26 (the ABI the ROS 2 binaries were built against) in force, and the failure that an unconstrained `pip install -U numpy` causes was reproduced in a throw-away copy of the setup; (ii) the wired RNDIS link has a mean ping of 0.45 ms against 28.3 ms over the access point, and the telemetry stream is evenly spaced over USB (102.3±8.8 ms) but arrives in bursts over Wi-Fi (120.2 ± 52.3 ms); (iii) the ROS 2 driver publishes `/odom` and `/joint_states` at about 10 Hz over the access point, and the full TF tree was inspected; (iv) the mecanum forward and inverse kinematic matrices were derived and checked numerically. Items that are still open are marked in the text.

Summary of results

Quantity	Wired USB (`rndis`)	Access point (`ap`)
Ping RTT, $N = 100$: mean / 95th pct / max	0.45 / 0.56 / 2.75 ms	28.3 / 127 / 152 ms
Ping loss	0 %	0 %
Telemetry arrival interval (mean ± std)	102.3 ± 8.8 ms	120.2 ± 52.3 ms
Key press to wheel response (mean / max)	0.16 / 0.30 s	0.19 / 0.50 s
SDK teleoperation log	299 rows, 29.9 s	299 rows, 29.9 s
ROS 2 `/odom` rate	n/a	9.8 Hz (4 to 226 ms)
ROS 2 bag	n/a	289.5 s, 5804 messages

Experimental setup

Hardware and software

Robot	DJI RoboMaster EP Core (Motion Controller and Intelligent Controller)
Host	Laptop, Ubuntu 24.04.5 LTS, kernel 7.0.0-38-generic, 233 GB SSD
ROS	ROS 2 Jazzy Jalisco (`ros-jazzy-desktop-full 0.11.0`), Gazebo Sim 8.15 (Harmonic)
Python	3.12.3; virtual environment `~/rm_venv`; NumPy 1.26.4; OpenCV 4.6.0
SDK	community fork `jeguzzi/RoboMaster-SDK` (`robomaster 0.1.2.0`)
ROS driver	`jeguzzi/robomaster_ros`, built with `colcon build -symlink-install`
Links	USB RNDIS (robot `192.168.42.2`), direct Wi-Fi hotspot (robot `192.168.2.1`)

The two physical topologies

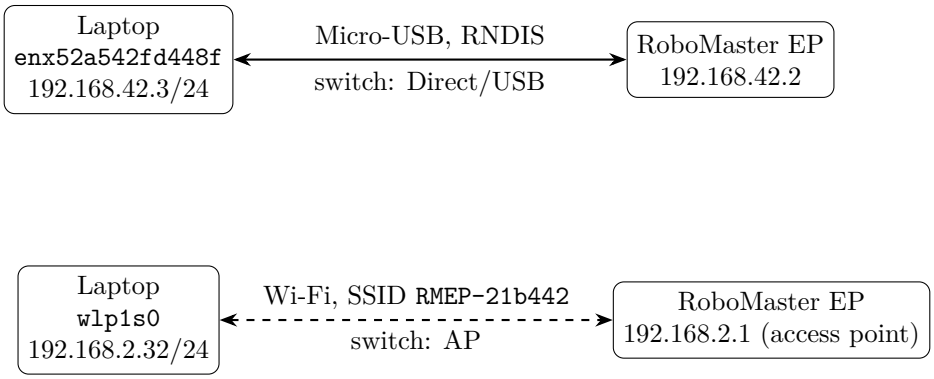


Figure 1: The two topologies used. Over USB the laptop keeps its normal Wi-Fi (and Internet); in AP mode the single Wi-Fi radio is tied to the robot, so the Internet connection is lost.

Method and data provenance

Every dataset was recorded by the scripts in the repository and saved unmodified: `ep_teleop_sdk.py` produced `logs/rndis_run.csv` and `logs/ap_run.csv`; `scripts/ping_bench.sh` produced the ping logs; `ros2 bag record` produced `logs/bag_ap`; `ros2 run tf2_tools view_frames` produced `figures/frames.pdf`. The analysis scripts (`analyze_telemetry.py`, `plot_odom.py`, `command_lag.py`, `make_report_figs.py`) only read these files. An earlier wired attempt was cut to 9.6 s by a USB enumeration failure (kernel message `device not accepting address, error -71`) and was repeated; only the complete runs are reported.

Problem 1: Host operating environment and middleware

(i) Operating system provisioning

Ubuntu 24.04 LTS runs bare-metal on the laptop (no virtual machine), so USB RNDIS and Wi-Fi packets are not subject to virtualization latency or loss. The system snapshot below was captured on the host.

```
ameen@ameen-G6: ~  (system diagnostics)

$ uname -srmo
Linux 7.0.0-38-generic x86_64 GNU/Linux

$ lsb_release -a
Distributor ID:    Ubuntu
Description:    Ubuntu 24.04.5 LTS
Release:    24.04
Codename:    noble

$ python3 --version
Python 3.12.3

$ echo ROS_DISTRO ; ros2 --help | head -1
jazzy
ros-jazzy-desktop-full 0.11.0-1noble.20261006.041807
ros-jazzy-ros-gz 1.0.24-1noble.20261006.041805

$ source ~/rm_setup.sh && python -c "import numpy ..."
/home/ameen/rm_venv/bin/python
numpy 1.26.4
numpy                              1.26.4
numpy-quaternion                   2024.0.13
PyYAML                             6.0.1
rm_libmedia_codec                  0.0.1
robomaster                         0.1.2.0

$ gz sim --versions
Gazebo Sim, version 8.15.0
Copyright (C) 2018 Open Source Robotics Foundation.

$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2       233G   30G  192G  14% /
```

Figure 2: Kernel, release, Python, ROS 2 and Gazebo versions, and disk space (`logs/system_diagnostics.txt`).

(ii) ROS 2 and simulation toolchain

ROS 2 Jazzy (`ros-jazzy-desktop-full`) and Gazebo Harmonic (`ros-jazzy-ros-gz`) are installed from the ROS apt repository together with the three development packages named in the assignment. Figure 3 shows the package versions and the workspace.

```
ameen@ameen-G6: ~  (installed packages and workspace)

$ ls ~/ros2_ws/src
robomaster_ros
$ colcon list --base-paths ~/ros2_ws/src
robomaster_description    /home/ameen/ros2_ws/src/robomaster_ros/robomaster_description    (ros.ament_cmake)
robomaster_msgs    /home/ameen/ros2_ws/src/robomaster_ros/robomaster_msgs    (ros.ament_cmake)
robomaster_ros    /home/ameen/ros2_ws/src/robomaster_ros/robomaster_ros    (ros.ament_python)
$ ls ~/ros2_ws/install
COLCON_IGNORE
local_setup.bash
local_setup.ps1
local_setup.sh
_local_setup_util_ps1.py
_local_setup_util_sh.py
local_setup.zsh
robomaster_description
robomaster_msgs
robomaster_ros
$ dpkg -l | grep -E "ros-jazzy-(desktop-full|ros-gz )"
ii ros-jazzy-desktop-full 0.11.0-1noble.20261006.041807
ii ros-jazzy-ros-gz 1.0.24-1noble.20261006.041805
$ dpkg -l | grep -E "libopus-dev|python3-colcon-common-extensions|python3-pip "
ii libopus-dev:amd64 1.4-1build1
ii python3-colcon-common-extensions 0.3.0-100
ii python3-pip 24.0+dfsg-1ubuntu1.3
```

Figure 3: Installed ROS 2, Gazebo and development packages, and the source workspace (`logs/captures/install_check.txt`).

(iii) Isolated Python virtual environment

The environment was created with `python3 -m venv -system-site-packages ~/rm_venv`. The option matters: ROS 2 ships its Python bindings (`rclpy`, `cv_bridge`) as binary extensions installed by apt, and a venv without system site-packages could not import them. The SDK dependencies were then installed *inside* the venv with `python -m pip`, which respects PEP 668 because the system interpreter is never modified:

```
python -m pip install -i https://test.pypi.org/simple/ rm-libmedia-codec
python -m pip install "numpy<2" "numpy-quaternion" pyyaml
python -m pip install git+https://github.com/jeguzzi/RoboMaster-SDK.git
```

Figure 4 shows what the environment resolves to after sourcing the loader script (Appendix B).

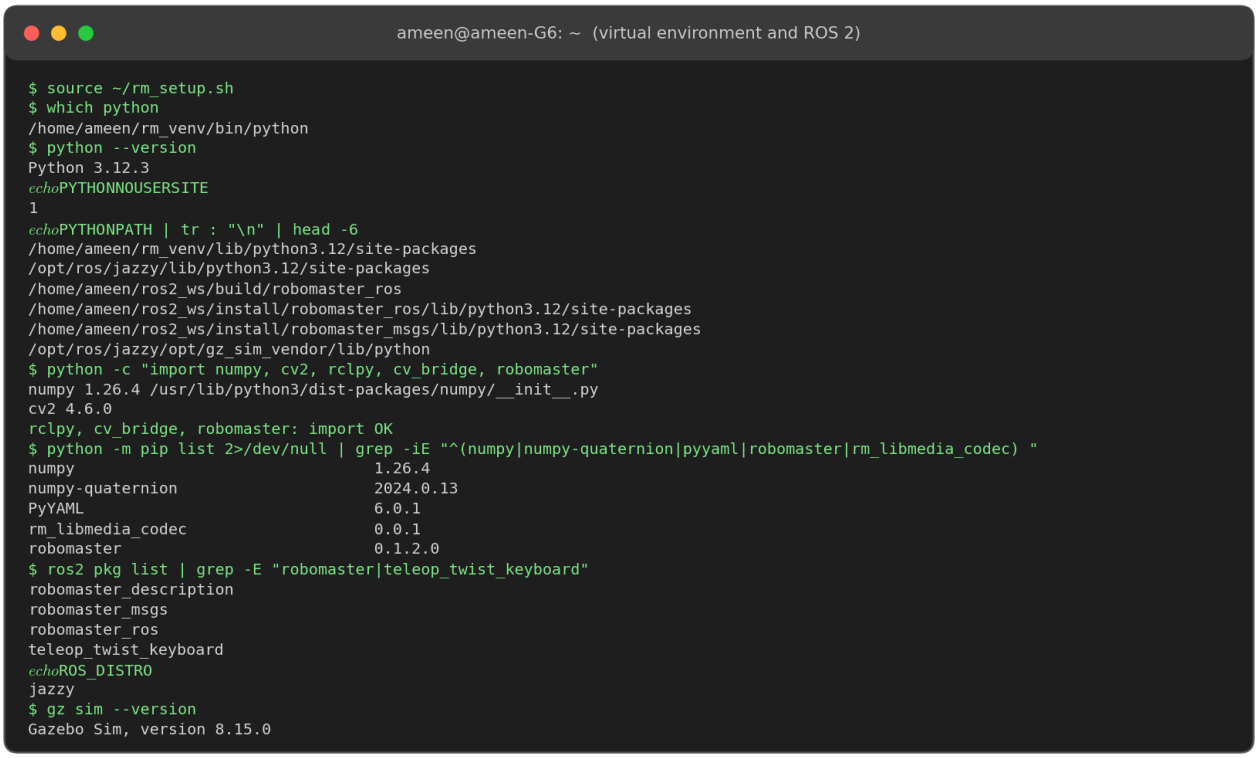


Figure 4: After `source ~/rm_setup.sh`: the interpreter is the venv's, `PYTHONNOUSERSITE=1`, the venv comes first on `PYTHONPATH`, and `numpy`, `cv2`, `rclpy`, `cv_bridge` and `robomaster` all import (`logs/captures/env_check.txt`).

(iv) ROS 2 driver compilation

The driver was cloned into `~/ros2_ws/src` and built with `colcon build -symlink-install`. The workspace contains the three packages `robomaster_ros` (Python), `robomaster_msgs` and `robomaster_description` (both CMake), and they appear in the output of `ros2 pkg list` (Figure 4) and `colcon list` (Figure 3). `-symlink-install` links the Python sources into `install/`, so edits to the driver take effect without a rebuild.

(v) ABI incompatibility analysis

What changed in NumPy 2.0. NumPy 2.0 is the first release in more than a decade to break the C ABI. Among other changes, struct layouts that extensions read directly (for example `PyArray_Descr`, whose element-size and alignment members were widened and whose legacy members were removed) changed, many deprecated macros and functions were removed, and the table

of function pointers behind `PyArray_API`, through which every compiled extension calls NumPy, was reorganised. The ABI version number checked at import time was raised accordingly.

Why a compiled module cannot survive that. A compiled extension does not call NumPy by name. At import it obtains the table of function pointers and thereafter calls NumPy by *index*, and it reads array and descriptor fields at *byte offsets* fixed when it was compiled. The ROS 2 binaries on this host (OpenCV's `cv2`, the Boost.Python module of `cv_bridge`, the converters used by `rclpy`) were compiled against NumPy 1.26 (Ubuntu 24.04 ships 1.26.4). If NumPy 2.x is what Python finds at run time, those indices and offsets point at the wrong slots and fields. The result is either a clean failure, where the module's import-time check raises `ImportError: A module that was compiled using NumPy 1.x cannot be run in NumPy 2.x`, or, when a mismatched access is not caught, a wrong function is called and the interpreter dies with a segmentation fault. Pure-Python code is unaffected, which is why the failure shows up only at the C boundary, for example when converting images.

Reproducing the failure safely. To show this rather than assert it, the setup was copied into a throw-away venv (`-system-site-packages`, outside the working environment), `pip install -U numpy` was run there, and the same imports were attempted. Figure 5 is the real output: pip installs NumPy 2.5.4, and importing the system `cv2` and `cv_bridge` fails with the message quoted above. (pip also printed a resolver warning that the system SciPy 1.11.4 requires `numpy<1.28`.)

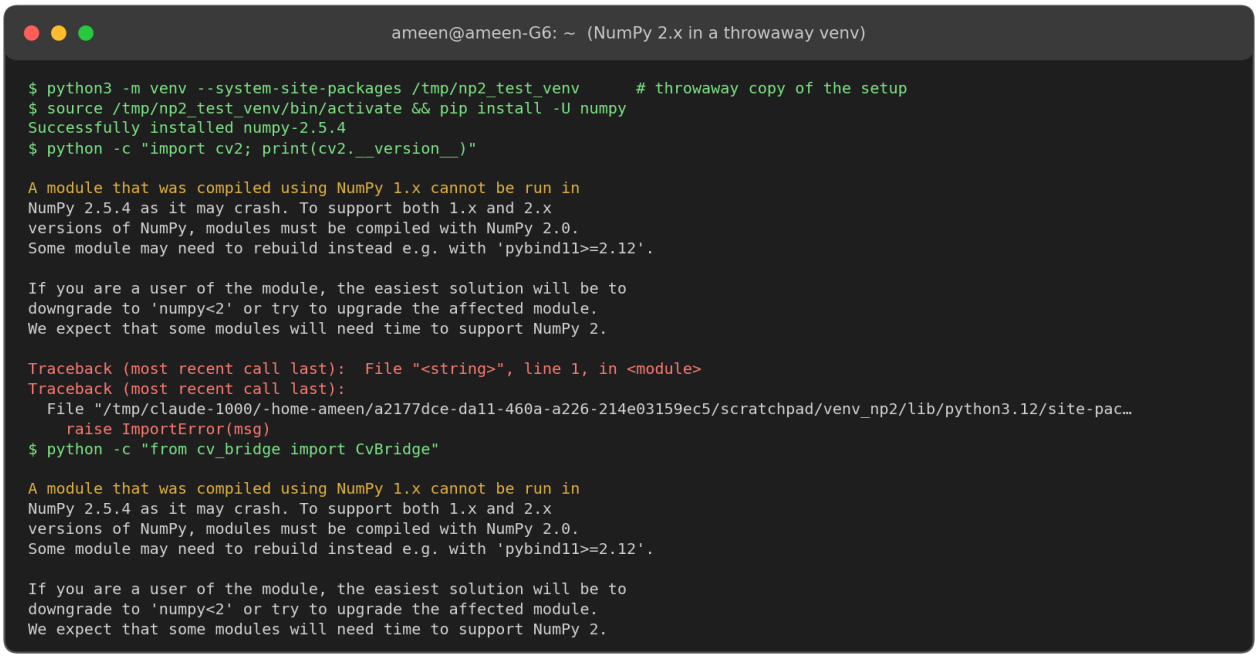


Figure 5: Real failure caused by an unconstrained NumPy upgrade in a throw-away copy of the setup (`logs/captures/numpy2_abi_failure.txt`). The working environment, `~/rm_venv`, was not touched.

Why the venv and the loader script prevent it. Because the venv is created with `-system-site-packages`, an unconstrained upgrade installs the new NumPy into the venv's own `site-packages`, which comes *first* on `sys.path` and therefore shadows the apt NumPy that the binaries need. The same shadowing occurs for a copy in `~/.local`. The setup defends against both:

1. **Pin:** `"numpy<2"` makes pip refuse any 2.x resolution inside the venv (the SDK's dependency `numpy-quaternion` would otherwise pull the latest release). On this host the pin is already satisfied by the system 1.26.4, so pip installs no second copy; `numpy.__file__` in Figure 4 points to `/usr/lib/python3/dist-packages`.

2. **Isolation:** the venv keeps all `pip` installs away from the system Python that PEP 668 protects.
3. **Search-path control:** `rm_setup.sh` sources ROS 2 and the workspace, activates the venv, prepends the venv's `site-packages` to `PYTHONPATH`, and exports `PYTHONNOUSERSITE=1` so packages in `~/.local` can never shadow anything. Every shell therefore resolves the same, ABI-consistent NumPy (the lines of `PYTHONPATH` in Figure 4).

The remedy if NumPy 2 is ever needed is the one the error message gives: recompile the affected extension against NumPy 2 headers, which produces a module that also runs on 1.x.

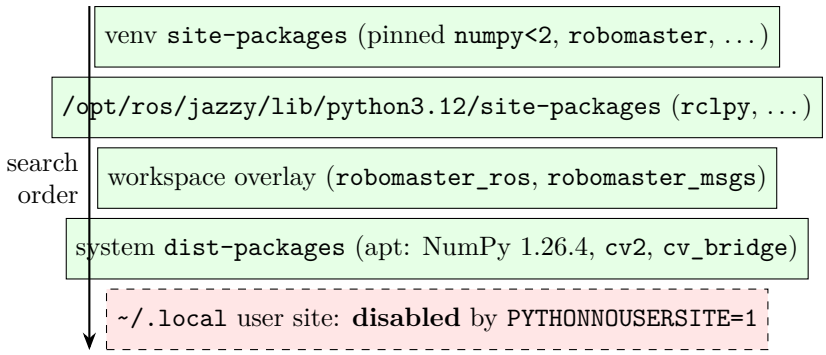


Figure 6: Python module search order after `source ~/rm_setup.sh`. Only one NumPy can be found, and it is the one the ROS 2 binaries were built for.

Problem 2: Telemetry and keyboard telecommand via the RoboMaster SDK

(i) Design of `ep_teleop_sdk.py`

The script (full listing in Appendix A) imports only `robomaster` and the Python standard library; it has no ROS 2 dependency. Its structure is shown in Figure 7.

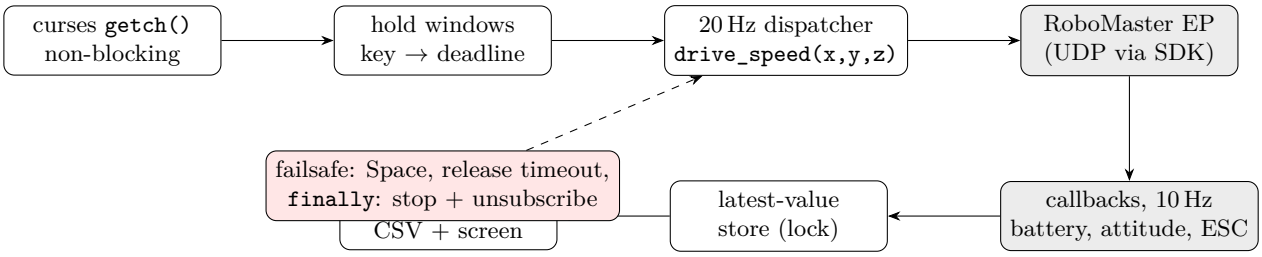


Figure 7: Architecture of the teleoperation and telemetry script.

Keyboard telecommand. The terminal is put in `curses` mode with `nodelay(True)`, so a read never blocks. W/S map to $v_x = \pm 0.3\,\mathrm{m/s}$, A/D to $v_y = \mp 0.3\,\mathrm{m/s}$ (the SDK's y axis points to the *right*, so D is $+y$) and Q/E to $\omega_z = \mp 45^\circ/\mathrm{s}$ (the SDK's yaw is *clockwise*). Commands are sent with `ep_robot.chassis.drive_speed()` at 20 Hz. A terminal delivers key *presses* and the operating system's auto-repeat, but never key *releases*. The script therefore gives each key a hold window: 0.55 s for the first press, which covers the typematic delay before auto-repeat starts, and 0.15 s once repeating. A key counts as held while the repeats keep arriving, and its contribution to the velocity vanishes when its window expires. Several keys can be combined, so forward plus strafe produces a diagonal.

Telemetry logging at 10 Hz. The script subscribes to `sub_battery_info`, `sub_attitude` and `sub_esc` at 10 Hz. The callbacks write into a lock-protected latest-value store, and a logger writes one CSV row every 100 ms. Besides the quantities asked for, each row records the arrival time of

the newest wheel-speed (ESC) packet, `t_esc_rx_unix`. This is what makes the network analysis of Section 3.4 possible, because the 10 Hz logger tick alone would only measure host scheduling.

Column	Unit	Meaning
`t_rel_s`, `t_unix`	s	time since start; wall-clock time of the row
`batt_pct`	%	battery state of charge
`yaw_deg, pitch_deg, roll_deg`	deg	chassis attitude (IMU)
`w_fr, w_fl, w_rl, w_rr_rpm`	rpm	four wheel speeds (SDK ESC order: FR, FL, RL, RR)
`t_esc_rx_unix`	s	arrival time of the newest ESC packet
`cmd_vx, cmd_vy, cmd_wz`	m/s, m/s, deg/s	command being sent when the row was written

Failsafe. Pressing Space zeroes all velocities and sends a stop command immediately, in the same loop iteration rather than at the next 20 Hz tick. Releasing the keys stops the chassis through the hold-window expiry. The cleanup code is in a `finally` block, so on *any* exit (duration reached, X pressed, Ctrl+C, or an exception) three stop commands are sent and the subscriptions are closed.

Software test before using the hardware. The control path was first exercised against a stub of the SDK inside a pseudo-terminal: holding W produced $v_x = 0.3$ m/s; W plus D combined; both fell to zero 0.45 s after the last key press; E produced 45°/s; Space zeroed all commands within 6 ms; and the logger wrote 10 rows/s. This verified the logic only and is not a hardware result.

(ii) Wired mode teleoperation (RNDIS)

The controller switch was set to Direct/USB and the robot connected through a Micro-USB data cable. Figure 8 shows the interface that appeared (`enx52a542fd448f`, `192.168.42.3/24`), the DJI device, and the ping to the robot's default RNDIS address.

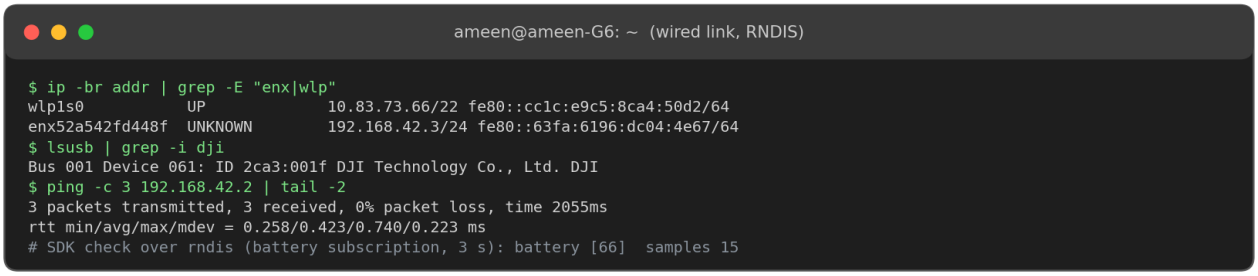


Figure 8: Wired link as seen from the host.

`python ep_teleop_sdk.py -conn rndis -duration 30 -out logs/rndis_run.csv` produced a complete 30 s run: 299 rows over 29.9 s with battery, attitude and all four wheel speeds and no missing values, and the newest ESC packet arrived 0.04 s before the file ended, so the link did not drop. Forward and lateral commands were used ($|v_x| = |v_y| = 0.3$ m/s); yaw was not, so the heading changed by only a few degrees of drift.

ep_teleop_sdk.py --conn rndis

```
$ python ep_teleop_sdk.py --conn rndis --duration 30 --out logs/rndis_run.csv
  (299 rows logged, columns shown: time, battery %, attitude deg, wheel rpm FR FL RL RR, command vx vy wz)

  t_rel_s   batt_pct   yaw_deg  pitch_deg  roll_deg  w_fr_rpm  w_fl_rpm  w_rl_rpm  w_rr_rpm  cmd_vx  cmd_vy  cmd_wz
    0.105     97.000    -6.010     -1.370    -0.230     3.000    -4.000     0.000    -4.000   0.000   0.000   0.000
    0.205     97.000    -6.010     -1.370    -0.230     0.000     1.000     1.000     0.000   0.000   0.000   0.000
    0.305     97.000    -6.010     -1.370    -0.230     0.000     1.000     2.000    -5.000   0.000   0.000   0.000
    0.405     97.000    -6.010     -1.370    -0.230    -3.000     3.000    -2.000    -1.000   0.000   0.000   0.000
    0.505     97.000    -6.010     -1.370    -0.230     0.000     6.000    -1.000    -3.000   0.000   0.000   0.000
    0.605     97.000    -6.010     -1.370    -0.230     1.000     4.000    -1.000    -1.000   0.000   0.000   0.000
      ...
   14.902     96.000    -4.770     -1.900    -0.540    76.000    67.000   -39.000   -55.000   0.000  -0.300   0.000
   15.002     96.000    -4.720     -1.850    -0.570    76.000    67.000   -39.000   -55.000   0.000  -0.300   0.000
   15.102     96.000    -4.710     -1.920    -0.610    65.000    41.000   -47.000   -48.000   0.000  -0.300   0.000
      ...
   29.603     96.000    -3.990     -1.460    -0.650    87.000   -71.000   -49.000    99.000   0.300   0.000   0.000
   29.703     96.000    -4.860     -1.710    -0.430    85.000   -67.000   -89.000    92.000   0.300   0.000   0.000
   29.804     96.000    -5.530     -1.780    -0.450    63.000   -59.000   -73.000    78.000   0.000   0.000   0.000
   29.904     96.000    -5.710     -1.790    -0.490    40.000   -46.000   -54.000    43.000   0.300   0.000   0.000
```

Figure 9: Terminal log of the wired run: first rows, rows from the middle, and last rows of `logs/rndis_run.csv`. The first row precedes the first wheel-speed packet, hence `nan`.

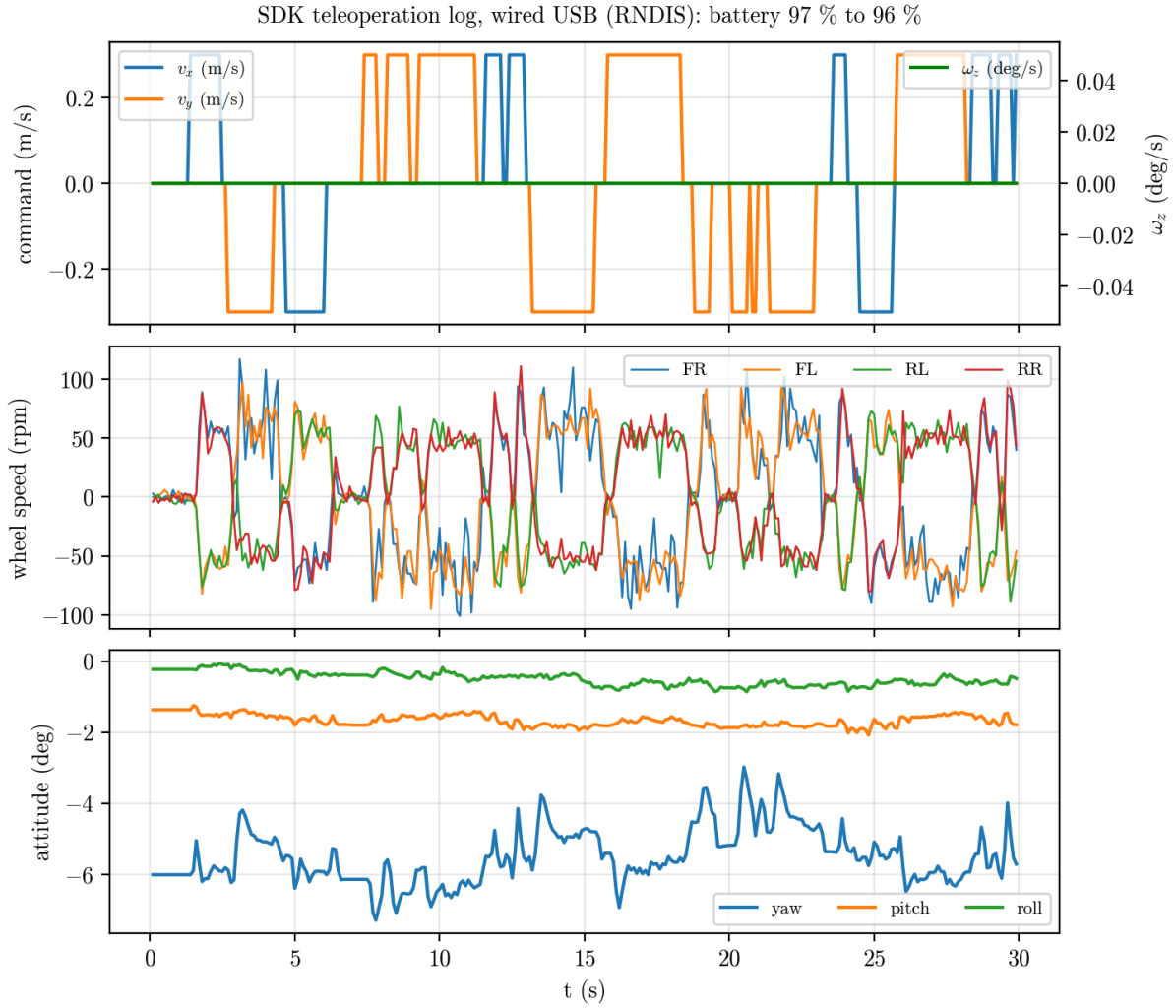


Figure 10: Wired run: commanded velocities, the four wheel speeds, and the attitude over the 30 s run.

(iii) Wireless hotspot teleoperation (AP)

With the switch on AP, the robot broadcast its own network (profile `RMEP-21b442`) and the laptop joined it. Figure 11 shows the resulting addressing: the laptop is `192.168.2.32/24` on `wlp1s0`, the route is the direct `192.168.2.0/24` subnet, and there is no gateway, which is why the Internet connection is lost in this mode. The last line is an error from my capture script (the `iw` tool is not

installed); it is kept because the capture is shown unedited.

ameen@ameen-G6: ~ (host network in AP mode)

```
# captured 2026-10-23T14:44:36+03:00
$ ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
wlp1s0           UP             192.168.2.32/24 fe80::b62f:2a42:7557:988d/64
$ ip route
192.168.2.0/24 dev wlp1s0 proto kernel scope link src 192.168.2.32 metric 600
$ nmcli -f GENERAL.CONNECTION,IP4.ADDRESS,IP4.GATEWAY dev show wlp1s0
GENERAL.CONNECTION:                     RMEP-21b442
IP4.ADDRESS[1]:                         192.168.2.32/24
IP4.GATEWAY:                            --
$ iw dev wlp1s0 link
/home/ameen/robomasters/assignment1/run_ap_all.sh: line 75: iw: command not found
```

Figure 11: Host addressing in AP mode (`logs/ap_host_info.txt`).

The AP run (`python ep_teleop_sdk.py -conn ap -duration 30 -out logs/ap_run.csv`) is also a complete 30 s run: 299 rows over 29.9 s, and this time all three motions were used (forward and lateral at 0.3 m/s, yaw at 45°/s), so the yaw changed from 90.3° to 122.0° (+31.7° for a turn command of about 0.7 s at 45°/s).

ep_teleop_sdk.py --conn ap

```
$ python ep_teleop_sdk.py --conn ap --duration 30 --out logs/ap_run.csv
  (299 rows logged, columns shown: time, battery %, attitude deg, wheel rpm FR FL RL RR, command vx vy wz)

  t_rel_s   batt_pct    yaw_deg  pitch_deg   roll_deg   w_fr_rpm   w_fl_rpm   w_rl_rpm   w_rr_rpm     cmd_vx     cmd_vy     cmd_wz
    0.100     49.000     90.300     -1.580      1.920        nan        nan        nan        nan      0.000      0.000      0.000
    0.201     49.000     90.300     -1.580      1.920     12.000      2.000      2.000      3.000      0.000      0.000      0.000
    0.301     49.000     90.300     -1.580      1.920      0.000      1.000      2.000      2.000      0.000      0.000      0.000
    0.402     49.000     90.300     -1.580      1.920     -1.000     -1.000      5.000     -1.000      0.000      0.000      0.000
    0.501     49.000     90.300     -1.580      1.920     -2.000      2.000      0.000     -7.000      0.000      0.000      0.000
    0.602     49.000     90.300     -1.580      1.920     -2.000      2.000      0.000     -7.000      0.000      0.000      0.000
      ...
   14.905     49.000     91.230     -1.580      1.830     64.000     58.000    -42.000    -47.000      0.000     -0.300      0.000
   15.001     49.000     91.160     -1.610      1.940     70.000     74.000    -54.000    -59.000      0.000     -0.300      0.000
   15.102     49.000     91.390     -1.580      1.900     96.000     65.000    -45.000    -50.000      0.000     -0.300      0.000
      ...
   29.600     49.000    121.960     -2.620      1.000      0.000      3.000      1.000     -2.000      0.000      0.000      0.000
   29.701     49.000    121.960     -2.620      1.000      0.000      0.000     -1.000     -2.000      0.000      0.000      0.000
   29.800     49.000    121.960     -2.620      1.010      1.000     -2.000     -2.000      0.000      0.000      0.000      0.000
   29.900     49.000    121.960     -2.620      1.010      0.000     -2.000     -4.000     -3.000      0.000      0.000      0.000
```

Figure 12: Terminal log of the AP run (`logs/ap_run.csv`).

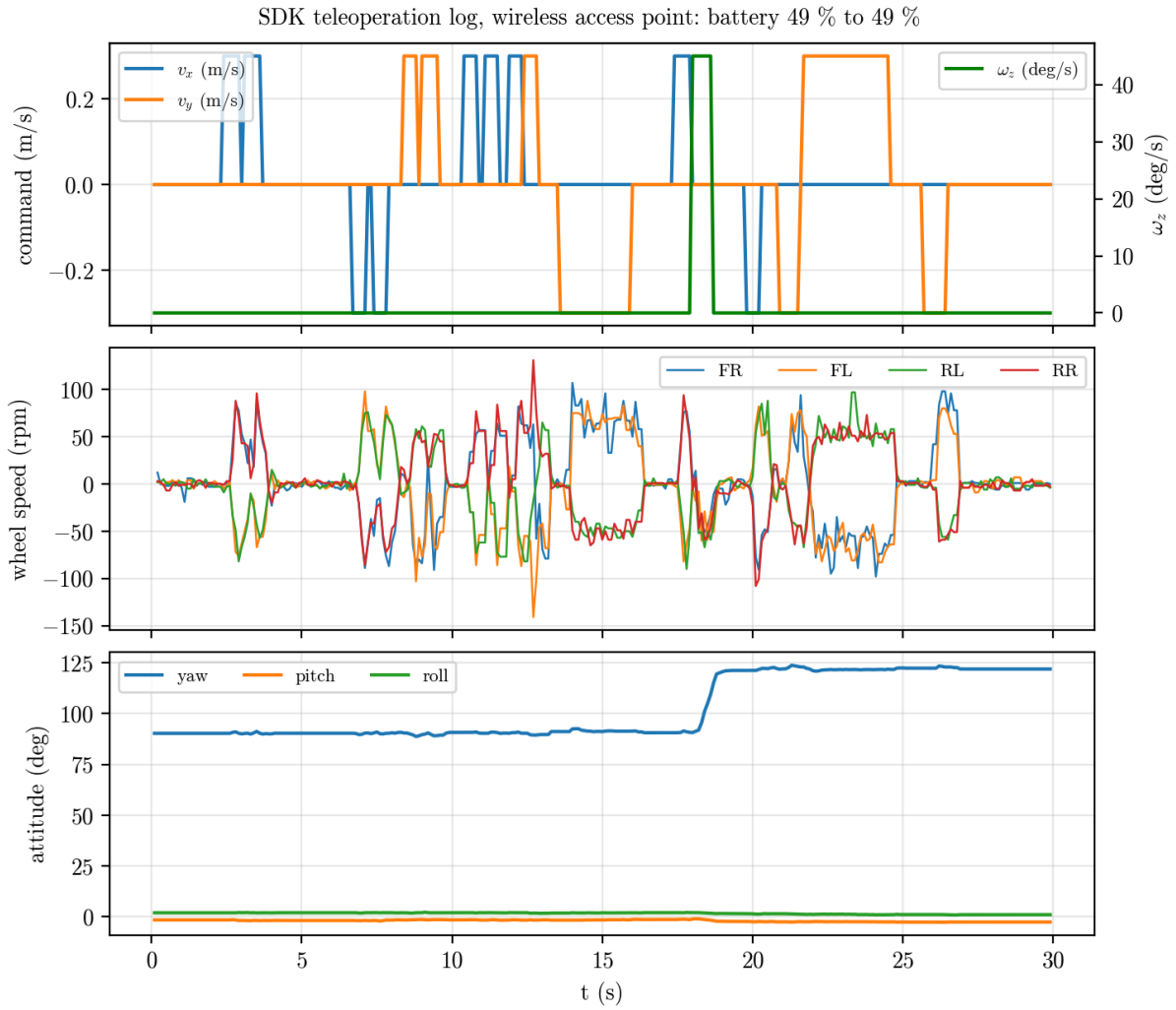


Figure 13: AP run: commands, wheel speeds and attitude. The logged ESC speeds are motor-signed: the left-side motors read opposite to the right-side ones for the same vehicle motion. After that sign convention is taken into account the patterns are those predicted by the kinematics of Problem 4 (see Table 2).

(iv) Comparative teleoperation latency analysis

Definitions. Ping round-trip times are from 100 ICMP packets 0.2 s apart. *Jitter* is reported two ways: the standard deviation of the RTT, and the mean absolute difference of successive RTTs, $\overline{|\mathrm{RTT}_i - \mathrm{RTT}_{i-1}|}$ (the RFC 3550-style estimate). The telemetry sample interval is $\Delta t = t_k - t_{k-1}$ computed on the *arrival times of distinct ESC packets* (network level), and for comparison also on the 10 Hz logger ticks (host scheduling only).

```
ameen@ameen-G6: ~  (ping, wired)

$ scripts/ping_bench.sh rndis
PING 192.168.42.2 (192.168.42.2) 56(84) bytes of data.
64 bytes from 192.168.42.2: icmp_seq=1 ttl=64 time=0.283 ms
64 bytes from 192.168.42.2: icmp_seq=2 ttl=64 time=0.375 ms
64 bytes from 192.168.42.2: icmp_seq=3 ttl=64 time=0.518 ms
64 bytes from 192.168.42.2: icmp_seq=4 ttl=64 time=0.352 ms
... 95 lines omitted ...
64 bytes from 192.168.42.2: icmp_seq=100 ttl=64 time=0.388 ms

--- 192.168.42.2 ping statistics ---
100 packets transmitted, 100 received, 0% packet loss, time 20564ms
rtt min/avg/max/mdev = 0.283/0.451/2.750/0.250 ms
```

Figure 14: Ping over the wired link (first and last lines of `logs/ping_rndis.txt`).

```
ameen@ameen-G6: ~  (ping, access point)

$ scripts/ping_bench.sh ap
PING 192.168.2.1 (192.168.2.1) 56(84) bytes of data.
64 bytes from 192.168.2.1: icmp_seq=1 ttl=64 time=60.5 ms
64 bytes from 192.168.2.1: icmp_seq=2 ttl=64 time=20.1 ms
64 bytes from 192.168.2.1: icmp_seq=3 ttl=64 time=21.9 ms
64 bytes from 192.168.2.1: icmp_seq=4 ttl=64 time=140 ms
... 95 lines omitted ...
64 bytes from 192.168.2.1: icmp_seq=100 ttl=64 time=4.54 ms

--- 192.168.2.1 ping statistics ---
100 packets transmitted, 100 received, 0% packet loss, time 19876ms
rtt min/avg/max/mdev = 1.680/28.255/152.066/36.476 ms
```

Figure 15: Ping over the access point (first and last lines of `logs/ping_ap.txt`).

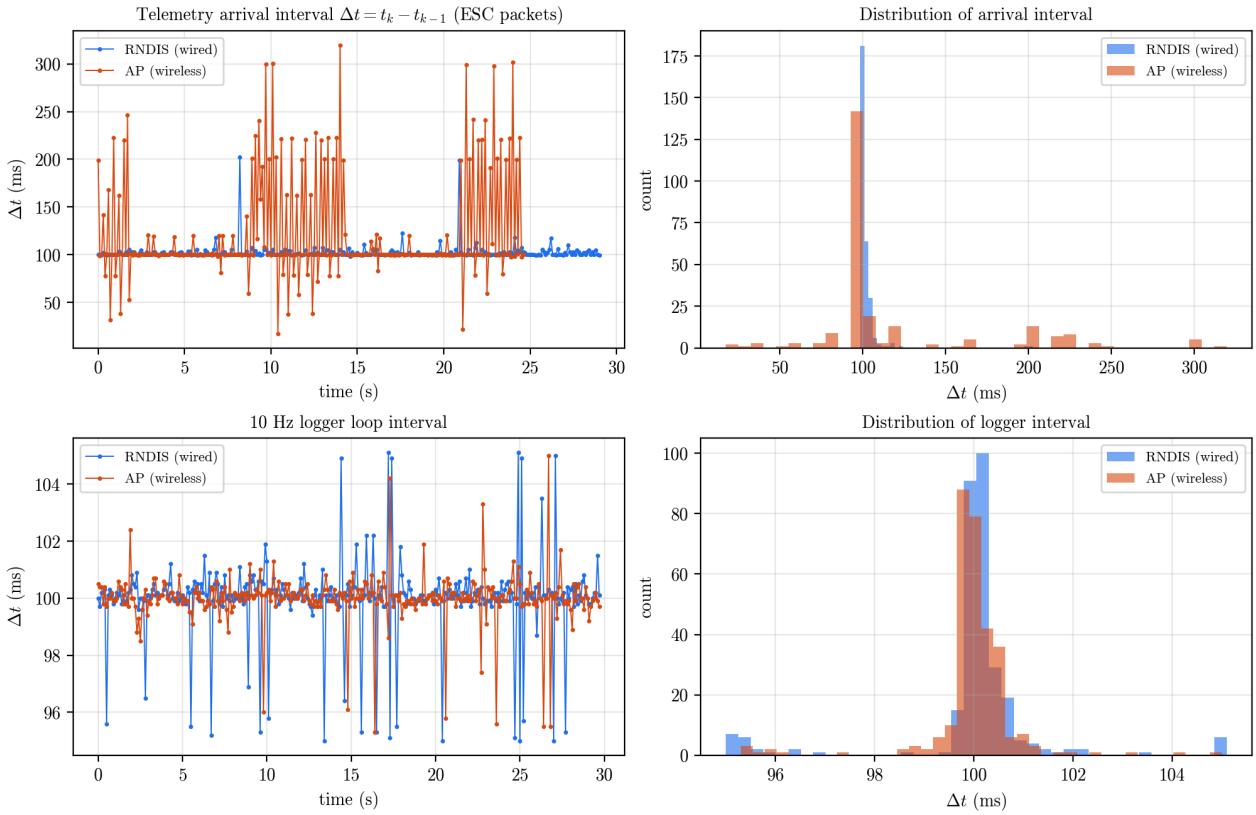


Figure 16: Telemetry sampling interval Δt. Top: arrival interval of ESC packets (network). Bottom: interval of the 10 Hz logger loop (host only; both links look alike, which confirms the difference above is due to the network and not to the laptop).

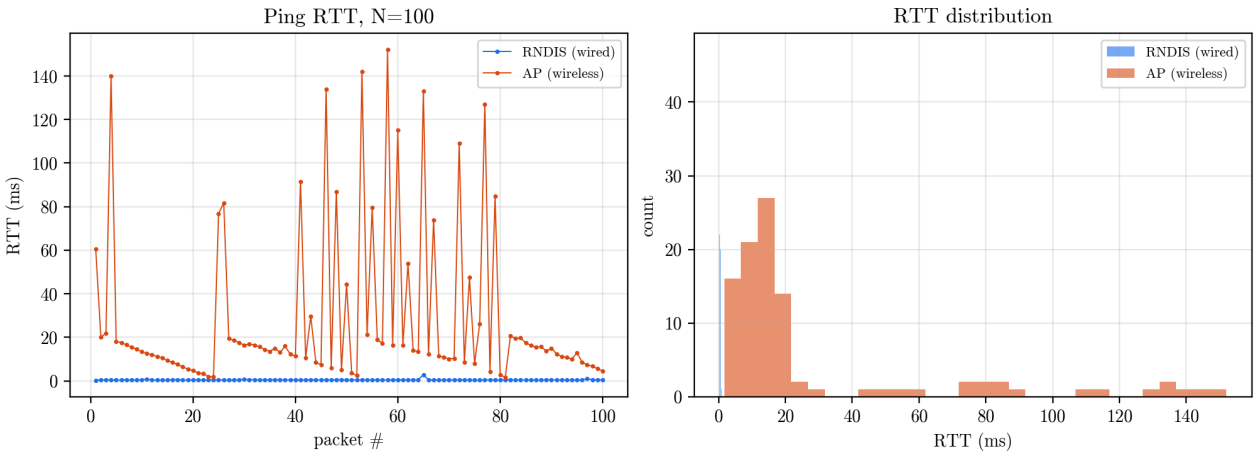


Figure 17: Ping RTT for the two links, $N = 100$.

	Wired RNDIS	**Wireless AP**
Ping loss ($N = 100$)	0 %	0 %
RTT mean (ms)	0.451	28.3
RTT std (ms)	0.252	36.7
Jitter, mean $\lvert \mathrm{RTT}_i - \mathrm{RTT}_{i-1} \rvert$ (ms)	0.137	32.1
RTT min / 95th pct / max (ms)	0.28 / 0.56 / 2.75	1.68 / 127.3 / 152.0
Telemetry Δt mean $\pm$ std (ms)	102.3 ± 8.8 (n=291)	120.2 ± 52.3 (n=247)
Telemetry Δt min / 95th pct / max (ms)	98.3 / 107.0 / 202.5	17.3 / 222.7 / 319.6
Key press to wheel response, mean / max (s)	0.16 / 0.30 (n=20)	0.19 / 0.50 (n=15)

Measured latency statistics. Both telemetry runs are complete 30 s logs. The response delays have a resolution of 0.1 s.

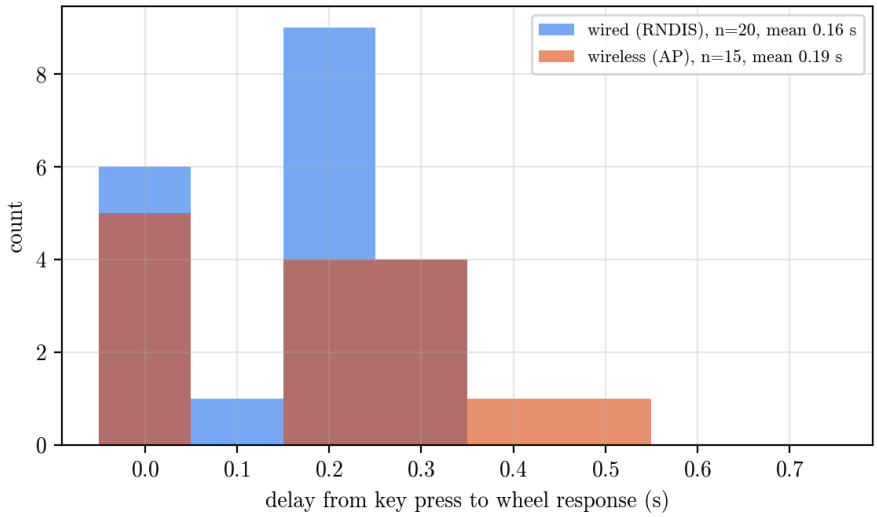


Figure 18: Delay between each key press and the first wheel response (mean absolute wheel speed above 15 rpm), from the two SDK logs.

Discussion.

- **Responsiveness (latency).** The mean RTT rises from 0.45 ms to 28.3 ms, about 63 times. The wired link never exceeded 2.8 ms; the access point reached 152 ms. USB RNDIS is a point-to-point link with no contention. A 2.4/5 GHz access point adds medium-access contention,

retransmission and radio power-saving wake-ups, which lift the mean and, more importantly, the tail.

- **Jitter.** The AP RTT has a standard deviation (36.7 ms) larger than its own mean, and a mean successive difference of 32.1 ms against 0.14 ms wired. The 95th percentile is 127 ms against 0.56 ms. The distribution is heavy-tailed rather than Gaussian, which is why the mean alone understates the wireless problem.
- **Telemetry intervals.** Wired packets arrive almost exactly every 100 ms (102.3 ± 8.8 ms; the maximum of 202.5 ms is one late packet). Over AP the stream is bursty (Figure 16): gaps of about 200 to 320 ms are followed by very short intervals (down to 17 ms) as delayed packets are delivered together. The mean stays near the nominal period (120 ms, since some packets are merged in the logger) while the standard deviation is about six times larger. The logger saw 248 distinct ESC packets in 299 ticks, so stale values were repeated during the delays, against 292 distinct packets over USB.
- **Command lag.** A command travels the one-way network delay and then waits for the next 20 Hz dispatch tick (up to 50 ms), and the robot has to accelerate the wheels. Measured from the logs, the delay between a key press and the first wheel response averaged 0.16 s wired and 0.19 s over AP, with worst cases of 0.30 s and 0.50 s. The means differ by less than the 0.1 s resolution, so the typical response is dominated by the robot's own acceleration and the hold-window logic; the wireless link shows up in the *tail*, a half-second worst case that matters for safety.
- **Safety implication.** With RTT spikes up to 152 ms, a key release or Space press can reach the robot about 0.15 s late; at 0.3 m/s that is roughly 4.5 cm of extra travel. This is why the failsafe is implemented as an immediate stop command plus an expiry of the hold window rather than relying on a single message.

Problem 3: Wireless teleoperation and telemetry in ROS 2

(i) Driver initialization over the hotspot

With the laptop on the robot's access point the driver was started with

```
source ~/rm_setup.sh
ros2 launch robomaster_ros ep.launch conn_type:=ap
```

The launch file starts three nodes: the driver `/robomaster`, `/robot_state_publisher` (from the robot description) and `/joint_state_publisher`. The driver log (Figure 19) shows `Try to connect via ap to robot with sn None` followed by `Connected`, the servo configuration of the arm, the list of enabled modules, and the (optional) FFmpeg warning, which only concerns an extra video transport.

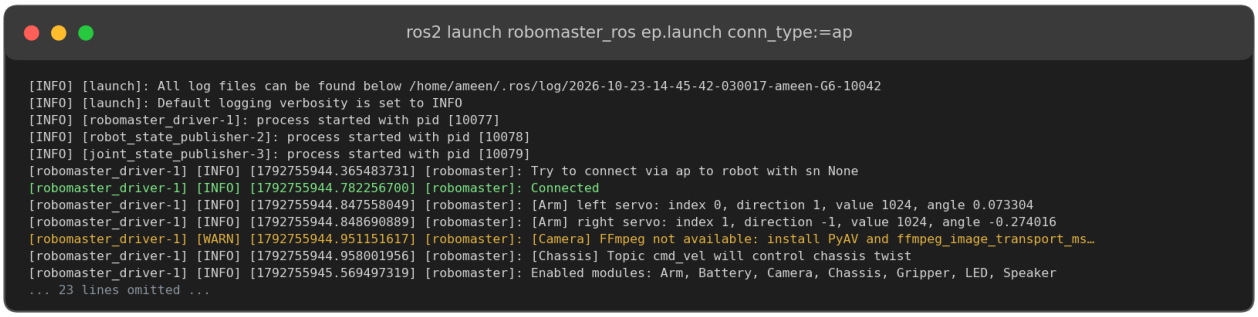


Figure 19: Start of the driver log over AP (`logs/ap_driver.log`).

Figure 20 is the verification the assignment asks for. `/cmd_vel` is advertised with one subscriber, the driver; `/odom`, `/joint_states`, `/imu`, `/battery`, `/tf` and the camera topics are published. The

measured rates over an 8 s window are:

- `/odom`: 9.8 Hz average, with intervals between 4 ms and 226 ms (standard deviation 46 to 51 ms);
- `/joint_states`: 10.2 Hz average, intervals between 19 ms and 101 ms (standard deviation 12 to 14 ms).

Both topics average the configured 10 Hz. The `/odom` intervals spread much wider, consistent with the odometry packets arriving over Wi-Fi in bursts as measured in Problem 2.

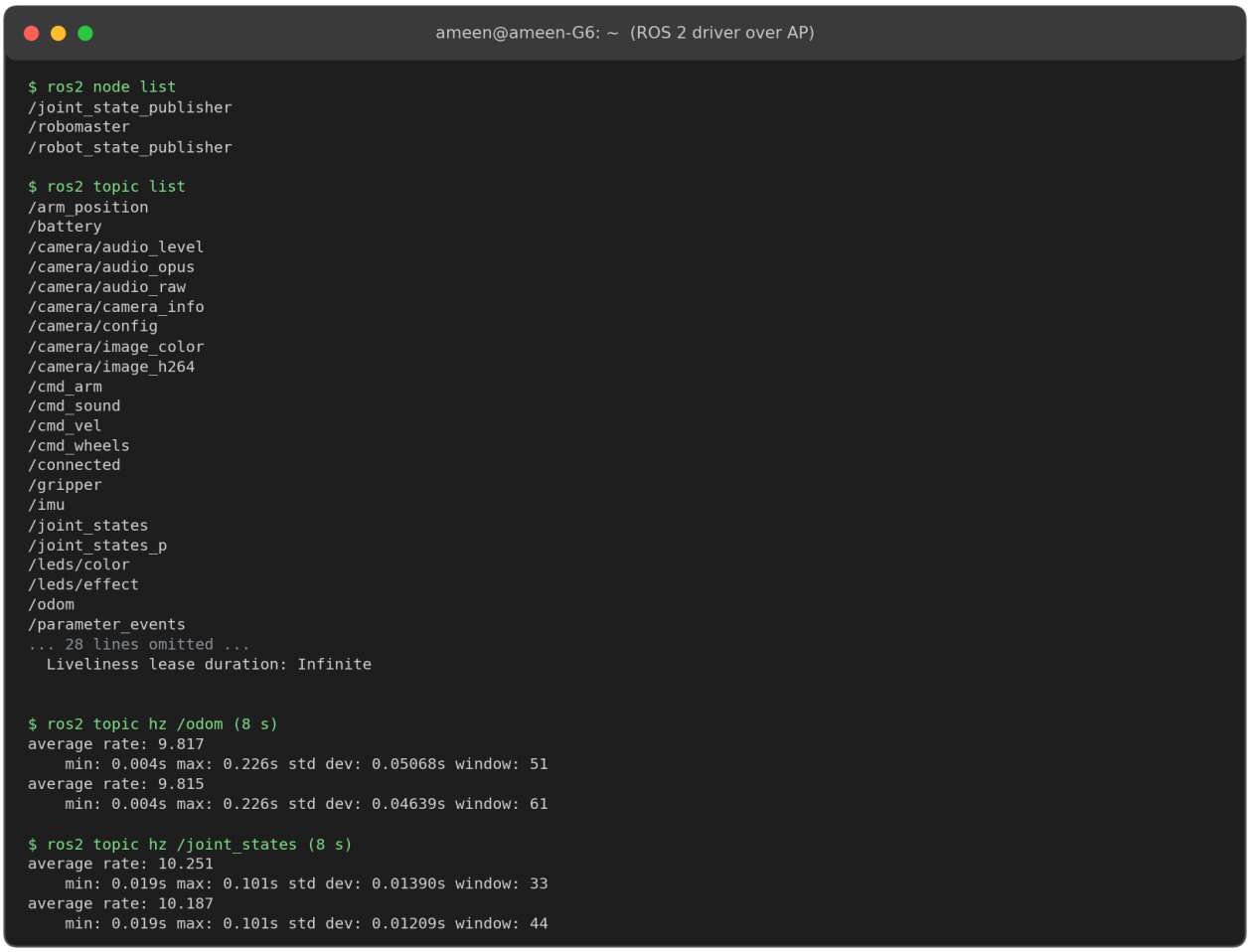


Figure 20: Nodes, topics, `/cmd_vel` endpoint and the measured publication rates (`logs/ap_ros_verification.txt`).

Where `/odom` comes from. The driver does not integrate velocities itself. In the chassis module it subscribes, through the SDK, to the robot controller's chassis position (`sub_position`), attitude, IMU, velocity and wheel speeds (`sub_esc`) and republishes them as `nav_msgs/Odometry`, `sensor_msgs/Imu` and the `odom` → `base_link` transform. The pose is therefore the motion controller's own dead-reckoned estimate, expressed in the frame the robot had when it booted.

(ii) Keyboard telecommand pipeline

In a second terminal:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args -r /cmd_vel:=/cmd_vel
```

The node publishes `geometry_msgs/Twist` on `/cmd_vel`: `i`/`,` forward/backward, `j`/`l` yaw, `k` stop, and Shift plus `j`/`l` (capitals) lateral strafing; the speed steps up and down with the `q`/`z` family of keys (the maximum linear command in the log is 0.55 m/s, i.e. 0.5×1.1). The recorded run lasted

289.5 s with 81 `/cmd_vel` messages (maximum $|v_x| = 0.55$ m/s, $|\omega_z| = 1.0$ rad/s), with roughly 21 s of forward or backward driving and 40 s of rotation. The path consists of straight legs joined by in-place turns (Figure 24), and the net heading change is +1447° (about four turns). **No lateral strafing was commanded in this Wi-Fi run** (max $|v_y| = 0$ in the bag). A second, shorter run was therefore recorded *over the USB link* (`conn_type:=rndis`, bag `logs/bag_usb`), with forward motion, Shift+J / Shift+L strafing and turning. It lasted 93.3 s with 918 odometry samples (9.8 Hz) and 46 `/cmd_vel` messages; the largest commands were $|v_x| = 0.5$ m/s, $|v_y| = 0.5$ m/s and $|\omega_z| = 1.0$ rad/s. Odometry gives a path length of 6.39 m, a net heading change of −53.5° and an end position of $(X, Y) = (-0.379, -0.117)$ m in the start frame (Figure 21). The tape-measured end position was not recorded for this run either, so the odometry error is still not quantified.

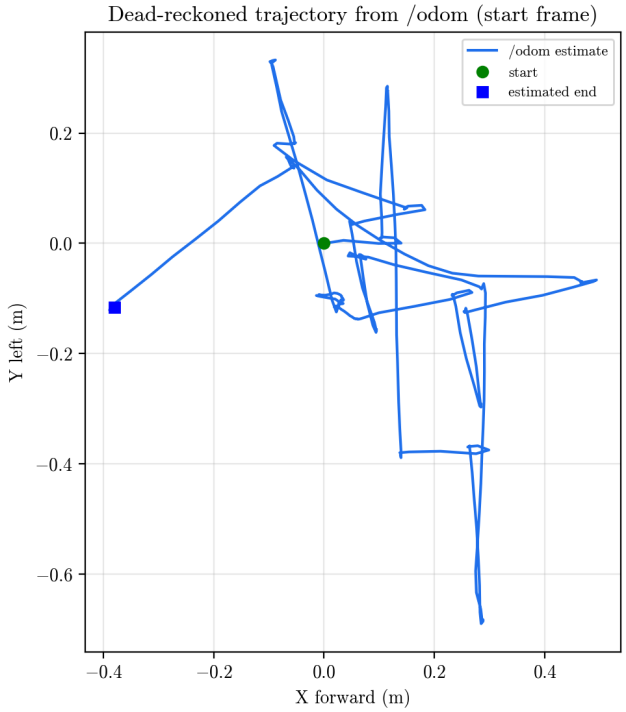


Figure 21: Odometry trajectory of the USB run with strafing (start frame, x forward, y left). No measured end position is available.

(iii) Telemetry acquisition

```
ros2 bag record -o logs/bag_ap /odom /joint_states /cmd_vel
```

```
ameen@ameen-G6: ~  (ros2 bag info)

$ ros2 bag info logs/bag_ap

Files:             bag_ap_0.mcap
Bag size:          5.2 MiB
Storage id:        mcap
ROS Distro:        jazzy
Duration:          289.501243996s
Start:             Oct 23 2026 15:47:07.777658130 (1792759627.777658130)
End:               Oct 23 2026 15:51:57.278902126 (1792759917.278902126)
Messages:          5804
Topic information: Topic: /cmd_vel | Type: geometry_msgs/msg/Twist | Count: 81 | Serialization Format: cdr
                   Topic: /joint_states | Type: sensor_msgs/msg/JointState | Count: 2896 | Serialization Format: cdr
                   Topic: /odom | Type: nav_msgs/msg/Odometry | Count: 2827 | Serialization Format: cdr
Service:           0
Service information:
```

Figure 22: Bag summary: 289.5 s, 5804 messages in MCAP format (`ros2 bag info logs/bag_ap`).

The bag holds 2827 `/odom` messages (9.78 Hz), 2896 `/joint_states` (10.0 Hz) and 81 `/cmd_vel` messages. The largest gap between consecutive `/odom` messages is 0.26 s and only three gaps exceed

0.25 s, so data were logged continuously without network stalls. Figure 23 compares the commanded velocities with the velocities reported in `/odom`: the reported velocity follows each command with a short delay and a smooth ramp, and the heading changes in steps at the in-place turns.

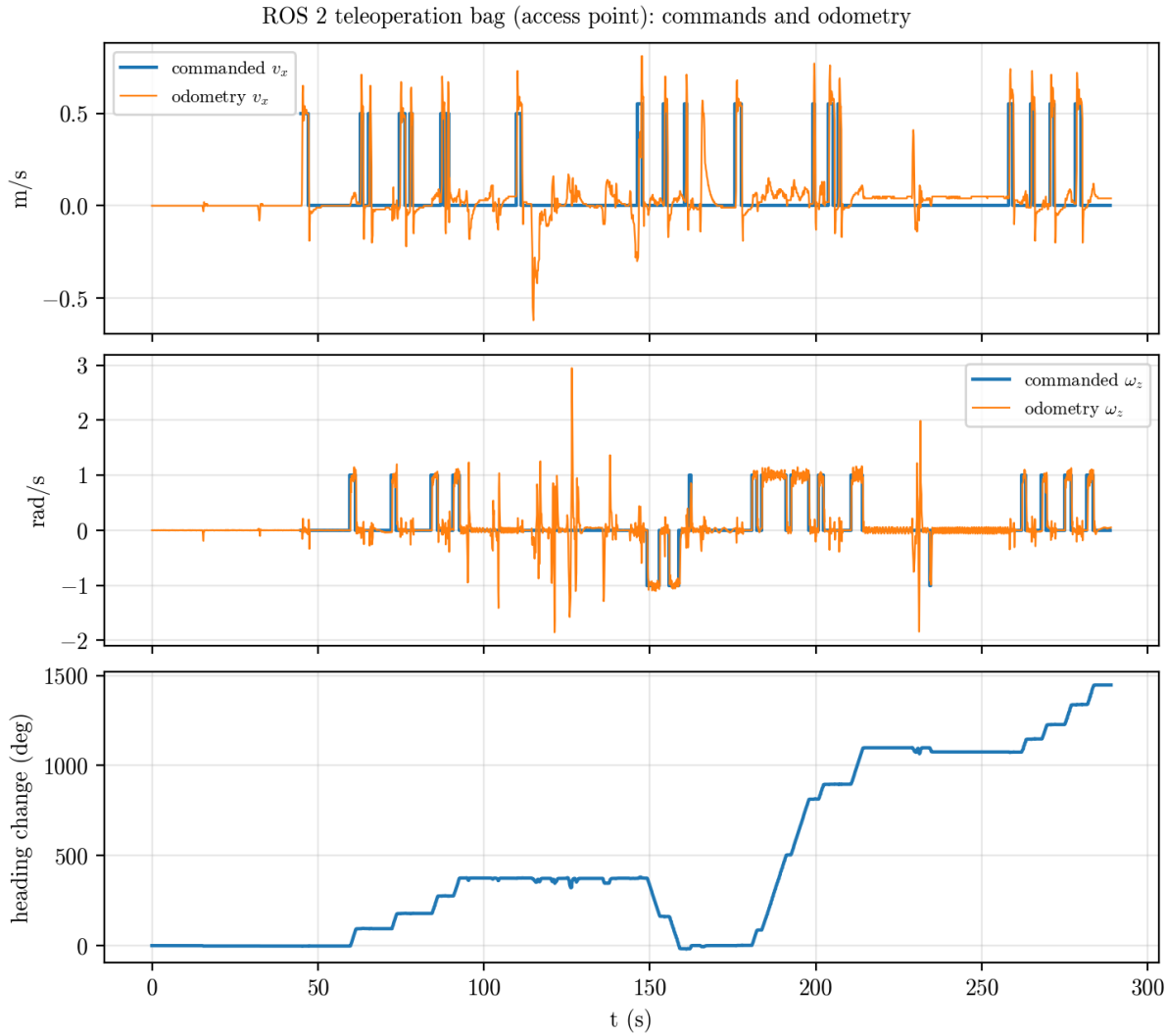


Figure 23: Commanded versus odometry velocities and the heading change, from the bag.

(iv) State estimation and odometry trajectory evaluation

The recorded `/odom` data were converted to the *start-pose frame* (origin at the robot's initial position, x along its initial heading, y to the left), because that is how a tape measurement from the start mark is expressed; `/odom` itself uses the boot frame, so the displacement is rotated by the initial yaw.

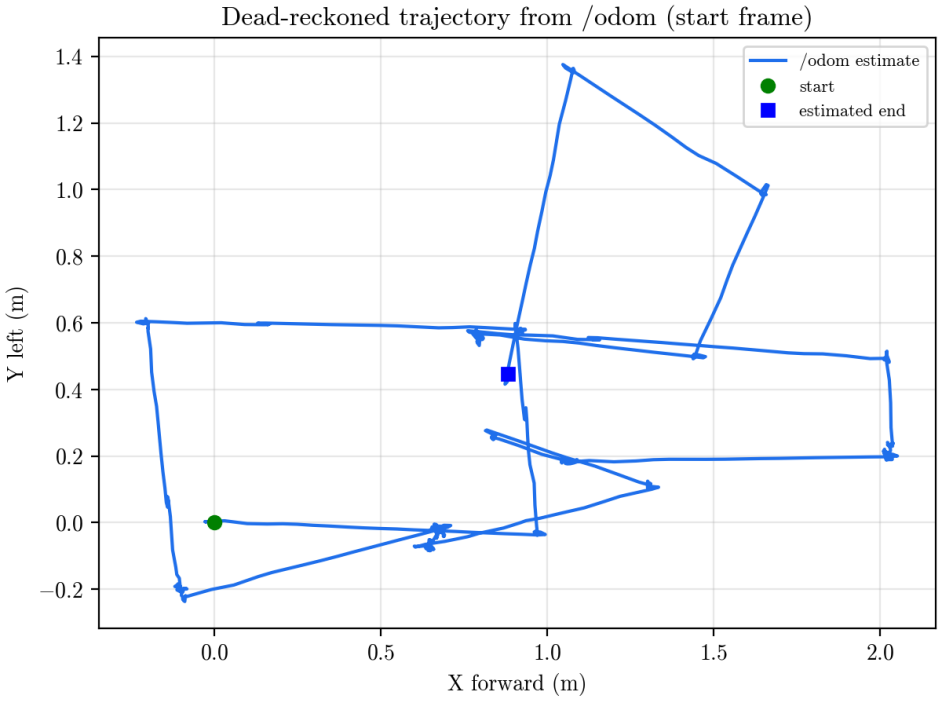


Figure 24: Dead-reckoned trajectory of the ROS 2 run in the start frame.

Samples / duration / rate	2827 / 288.9 s / 9.8 Hz
Integrated path length	14.86 m
Estimated end position (start frame)	$(X, Y) = (0.881,\ 0.447)$ m
Net heading change	+1447°
Largest sample-to-sample step	0.13 m (no jumps)
Tape-measured end position	**[TODO: not measured for this run]**

Endpoint comparison. The estimated end position is given above; the error against the true position, in metres and as a percentage of the path length, can only be filled in from a tape measurement taken at the end of a run.

Sources of odometry drift. Position is the running integral

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{R}(\psi_k)\,\mathbf{v}_k\,\Delta t,$$

and nothing ever corrects it, so its error grows with the distance travelled.

- **Dead-reckoning integration and heading error.** A heading error $\delta\psi$ rotates every later displacement, so it produces a lateral error of about $d\,\delta\psi$ after travelling a distance d. For this run: one degree of heading error (1.7×10^{-2} rad) already gives 1.7 cm of lateral error per metre, and 26 cm if it existed over the whole 14.9 m path. The run contains about four in-place turns, each of which can add a small heading error.
- **Mecanum roller slip.** The lateral and yaw motions of a mecanum platform rely on the rollers sliding at 45°, so wheel rotation is only loosely tied to ground displacement, and encoders measure the wheel, not the ground, so slip is invisible to odometry. The logs show a direct sign of it: in the steady parts of the strafe commands (D or A) the front wheels ran at about 62 to

69 rpm while the rear wheels ran at about 49 to 56 rpm in both the wired and AP logs (Table 2), although the kinematics predicts equal speeds of 57.3 rpm. Driving straight, all four wheels were within a few rpm of each other.

- **Model and discretization errors.** The nominal radius and geometry (R, l_x, l_y) differ from the real values (tyre wear, load), and sampling at 10 Hz with first-order integration ignores the motion inside a step during turns and accelerations.
- **Sensing noise and bias.** Gyro bias and encoder quantization add a slowly growing error, and a late or dropped `/odom` packet over Wi-Fi (gaps up to 0.26 s here) coarsens the integration step.

Problem 4: Theoretical formulation and system diagnostics

(i) Network topology characterization

Metric	Wired USB (`rndis`)	Direct hotspot (`ap`)
Host interface name	`enx52a542fd448f`	`wlp1s0`
Host IP and netmask	192.168.42.3/24	192.168.2.32/24
Robot endpoint IP	192.168.42.2	192.168.2.1
Round-trip latency (avg $\pm$ std, $N = 100$)	0.45 ± 0.25 ms (jitter 0.14 ms)	28.3 ± 36.7 ms (jitter 32.1 ms)
Host internet retained?	Yes (observed: the Wi-Fi stayed connected while the USB link was up).	No (observed: the AP profile has no gateway and the only Wi-Fi radio is tied to the robot).
Multi-robot scalability	Poor. One cable and one USB interface per robot, and every robot uses the same 192.168.42.2 subnet, so they collide on one host without extra isolation.	Poor. A host Wi-Fi adapter joins one AP at a time and every robot AP uses 192.168.2.1, so one adapter serves one robot.

Table 1: Physical connection topology comparison. All numbers are the measured ones of Problem 2 and the host captures of Figures 8 and 11.

Reading the table. The wired link is the better choice for development and for anything latency-sensitive: sub-millisecond RTT, no packet bursts, and the laptop keeps its Internet connection (useful for installing packages, or for talking to a remote tool while the robot is attached). Its drawbacks are the cable itself, which limits the robot's range and proved to be the weak point in practice (a marginal connection caused an enumeration failure during one attempt), and that every robot presents the same address. The hotspot gives untethered operation but trades latency (63 times higher RTT) and jitter (a heavy tail up to 152 ms) for it, loses Internet access on a single-radio laptop, and cannot scale to several robots from one adapter. Router-based operation (STA mode) would remove the multi-robot and Internet limits but depends on the building network and was excluded by the assignment.

(ii) Mecanum kinematic derivation

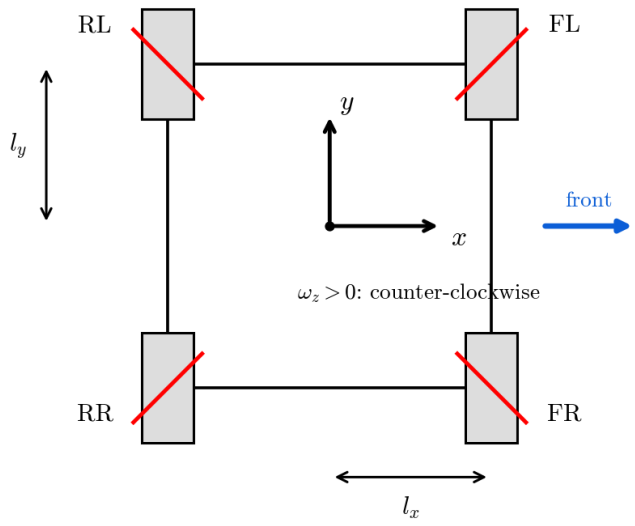


Figure 25: Geometry used in the derivation: body frame (x forward, y left, ω_z counter-clockwise), half-wheelbase l_x, half-track l_y, and the assumed roller axes. This corrects the assignment's Figure 1 in one respect: there the wheel labels are drawn as if the robot faced up the page while the x axis points right; here the front is drawn explicitly.

Setup. Wheel i has radius R, angular velocity ω_i (positive drives the vehicle forward) and sits at (x_i, y_i) with $x_i = \pm l_x$ and $y_i = \pm l_y$: FL $= (+l_x, +l_y)$, FR $= (+l_x, -l_y)$, RL $= (-l_x, +l_y)$, RR $= (-l_x, -l_y)$. The rollers sit at angle $\gamma_i = \pm 45°$ to the wheel's driving direction (the standard RoboMaster layout: FL and RR have one roller handedness, FR and RL the other). The figure in the assignment does not draw rollers, so this layout is an assumption, and it is checked against the measured wheel patterns below.

Step 1: velocity of the contact point. For the body twist $\mathbf{v}_b = [v_x, v_y, \omega_z]^T$ the velocity of the point at (x_i, y_i) is

$$\mathbf{v}_i = \begin{bmatrix} v_x - \omega_z y_i \\ v_y + \omega_z x_i \end{bmatrix}.$$

Step 2: wheel and roller constraint. The contact point moves with the rim speed $u_i = R\omega_i$ along the wheel's driving direction $\hat{\mathbf{x}}$, plus a free sliding speed s_i along the roller axis $\mathbf{r}_i = (\cos\gamma_i, \sin\gamma_i)$:

$$\mathbf{v}_i = u_i \hat{\mathbf{x}} + s_i \mathbf{r}_i.$$

The y component gives $s_i = v_{iy} / \sin\gamma_i$. Substituting into the x component:

$$R\omega_i = u_i = v_{ix} - v_{iy} \cot\gamma_i, \qquad \sigma_i := \cot\gamma_i = \pm 1.$$

Step 3: insert the contact velocity.

$$R\omega_i = v_x - \sigma_i v_y - \omega_z (y_i + \sigma_i x_i).$$

With $\sigma = (+1, -1, -1, +1)$ for (FL, FR, RL, RR), every term $y_i + \sigma_i x_i$ equals $\pm(l_x + l_y)$. Defining $k = l_x + l_y$:

$$\omega_{fl} = \tfrac{1}{R}(v_x - v_y - k\,\omega_z), \quad \omega_{fr} = \tfrac{1}{R}(v_x + v_y + k\,\omega_z),$$
$$\omega_{rl} = \tfrac{1}{R}(v_x + v_y - k\,\omega_z), \quad \omega_{rr} = \tfrac{1}{R}(v_x - v_y + k\,\omega_z).$$

Inverse kinematics.

$$\begin{bmatrix}\omega_{fl}\\ \omega_{fr}\\ \omega_{rl}\\ \omega_{rr}\end{bmatrix} = \mathbf{J}_{IK}\begin{bmatrix}v_x\\ v_y\\ \omega_z\end{bmatrix}, \qquad \mathbf{J}_{IK} = \frac{1}{R}\begin{bmatrix}1 & -1 & -k\\ 1 & 1 & k\\ 1 & 1 & -k\\ 1 & -1 & k\end{bmatrix}.$$

Forward kinematics. $\mathbf{J}_{IK}$ is 4×3 with full column rank, so the least-squares inverse is the pseudo-inverse $\mathbf{J}_{FK} = (\mathbf{J}_{IK}^T\mathbf{J}_{IK})^{-1}\mathbf{J}_{IK}^T$. The three columns $\mathbf{c}_1 = (1,1,1,1)$, $\mathbf{c}_2 = (-1,1,1,-1)$, $\mathbf{c}_3 = (-k,k,-k,k)$ (up to the factor $1/R$) are mutually orthogonal: $\mathbf{c}_1\cdot\mathbf{c}_2 = 0$, $\mathbf{c}_1\cdot\mathbf{c}_3 = 0$, $\mathbf{c}_2\cdot\mathbf{c}_3 = k + k - k - k = 0$, with $\|\mathbf{c}_1\|^2 = \|\mathbf{c}_2\|^2 = 4$ and $\|\mathbf{c}_3\|^2 = 4k^2$. Hence

$$\mathbf{J}_{IK}^T\mathbf{J}_{IK} = \frac{1}{R^2}\,\mathrm{diag}(4,\,4,\,4k^2),$$

$$\begin{bmatrix}v_x\\ v_y\\ \omega_z\end{bmatrix} = \mathbf{J}_{FK}\begin{bmatrix}\omega_{fl}\\ \omega_{fr}\\ \omega_{rl}\\ \omega_{rr}\end{bmatrix}, \qquad \mathbf{J}_{FK} = \frac{R}{4}\begin{bmatrix}1 & 1 & 1 & 1\\ -1 & 1 & 1 & -1\\ -\frac{1}{k} & \frac{1}{k} & -\frac{1}{k} & \frac{1}{k}\end{bmatrix}.$$

Checks. $\mathbf{J}_{FK}\mathbf{J}_{IK} = \mathbf{I}_3$: for example the $(1,1)$ entry is $\frac{R}{4}\cdot\frac{1}{R}(1+1+1+1) = 1$ and the $(3,3)$ entry is $\frac{R}{4k}\cdot\frac{1}{R}(k+k+k+k) = 1$. The product the other way, $\mathbf{J}_{IK}\mathbf{J}_{FK}$, is only a rank-3 projector and not $\mathbf{I}_4$: four wheels span a 3-D twist space, and the left-over wheel-speed direction is $\mathbf{n} = \frac{1}{2}(1,1,-1,-1)$ in the order (FL, FR, RL, RR), i.e. the two front wheels forward and the two rear wheels backward, which produces no body motion and is how wheels "fight" each other and cause slip. The numerical check below confirms $\mathbf{J}_{FK}\mathbf{n} = \mathbf{0}$.

Numbers. Nominal RoboMaster EP values: $R = 0.05\,\mathrm{m}$, $l_x = l_y = 0.10\,\mathrm{m}$, hence $k = 0.20\,\mathrm{m}$. Then

$$\mathbf{J}_{IK} = \begin{bmatrix}20 & -20 & -4\\ 20 & 20 & 4\\ 20 & 20 & -4\\ 20 & -20 & 4\end{bmatrix}, \qquad \mathbf{J}_{FK} = \begin{bmatrix}0.0125 & 0.0125 & 0.0125 & 0.0125\\ -0.0125 & 0.0125 & 0.0125 & -0.0125\\ -0.0625 & 0.0625 & -0.0625 & 0.0625\end{bmatrix},$$

and the singular values of $\mathbf{J}_{IK}$ are $40,\ 40,\ 8$, so the matrix is well conditioned (condition number 5). The nominal dimensions should be confirmed against the datasheet.

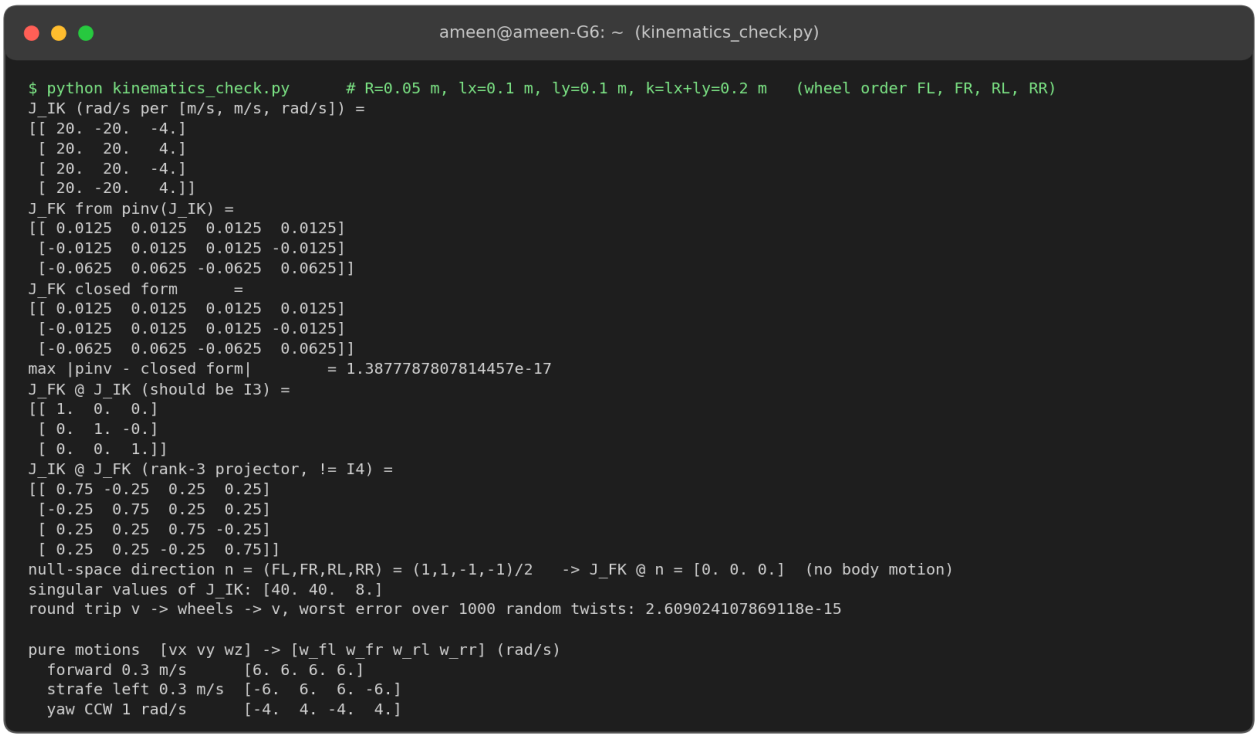


Figure 26: Numerical check of the derivation (`scripts/kinematics_check.py`): the pseudo-inverse equals the closed form to 10^{-17}, $\mathbf{J}_{FK}\mathbf{J}_{IK} = \mathbf{I}_3$, the null direction is $(1, 1, -1, -1)/2$, and 1000 random twists round-trip with error below 3×10^{-15}.

Comparison with the measured wheel speeds. The prediction was checked against the logs of Problem 2. Table 2 lists the mean ESC speeds in the steady part of single-key commands. The ESC speeds are *motor-signed*: the left-side motors (FL and RL) read opposite to the right-side ones for the same vehicle motion, e.g. driving forward gives +54 and +55 rpm on FR and RR but −55 and −60 rpm on FL and RL. After flipping the sign of the left wheels the logged order (FR, FL, RL, RR) reproduces the predicted pattern for both forward $(+, +, +, +)$ and strafing (FR −, FL +, RL −, RR + for D), which confirms the assumed roller layout and wheel order. The forward magnitude (54 to 60 rpm) is within 6 % of the predicted $v/R = 57.3$ rpm.

Log	**Steady command**	**FR**	**FL**	**RL**	**RR** (rpm)
prediction (sign-corrected, 57.3 rpm)	W	+57.3	+57.3	+57.3	+57.3
prediction	D (strafe right)	−57.3	+57.3	−57.3	+57.3
wired, raw ESC	W or S ($v_x > 0$)	+54.1	−54.6	−59.6	+54.9
wired, left sign flipped	W	+54.1	+54.6	+59.6	+54.9
wired, raw ESC	D	−61.8	−64.1	+51.2	+50.5
wired, left sign flipped	D	−61.8	+64.1	−51.2	+50.5
AP, raw ESC	D	−62.7	−65.3	+56.6	+52.4
AP, left sign flipped	D	−62.7	+65.3	−56.6	+52.4

Table 2: Predicted versus measured wheel speeds (means over rows at least 0.5 s into a single-key command; $n = 10$ to 58 rows each). The strafe magnitudes differ between front (62 to 65 rpm) and rear (51 to 57 rpm) wheels, an indication of roller slip.

(iii) ROS 2 coordinate transform inspection

`ros2 run tf2_tools view_frames` was run with the driver up. Figure 27 summarizes the tree and the full diagram follows on its own page. `ros2 topic info /tf -v` (Figure 28) lists exactly two

publishers of `/tf`: the driver node `/robomaster` and `/robot_state_publisher`.

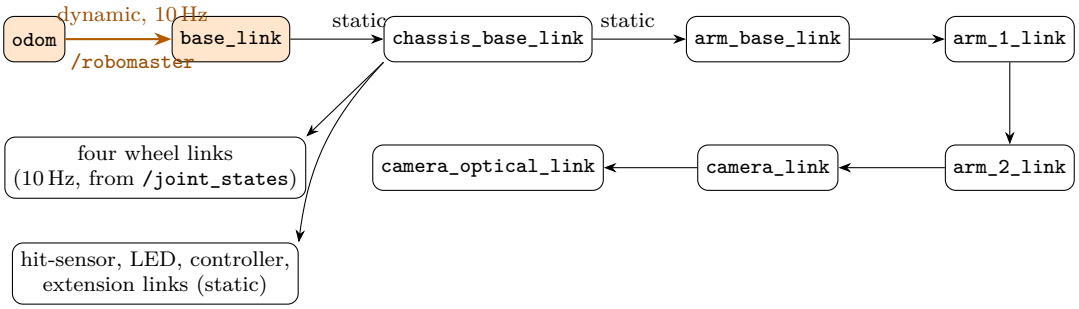


Figure 27: Condensed TF tree (orange: published by the driver; the rest by `robot_state_publisher` from the robot description and `/joint_states`).

Parent	Child	Type	Published by
`odom`	`base_link`	dynamic, 10.1 Hz	driver node `/robomaster` (chassis module)
`base_link`	`chassis_base_link`	static	`/robot_state_publisher`
`chassis_base_link`	`arm_base_link`	static (fixed joint, offset (0, 0.0010, 0.0906) m)	`/robot_state_publisher`
`chassis_base_link`	four wheel links	dynamic, 10.2 Hz	`/robot_state_publisher` (from `/joint_states`)
`arm_base_link`	`arm_1_link`, …	dynamic, 10.2 Hz	`/robot_state_publisher`

There is no direct `base_link` → `arm_base_link` transform: the arm is attached to the chassis through the chain `base_link` → `chassis_base_link` → `arm_base_link`. `odom` is the root of the tree. (An extra observation, used in Assessment 2: the robot description attaches the camera to the arm, but images show that the real camera does not move with the arm, so this part of the TF tree should not be used for geometry.)

ameen@ameen-G6: ~ (TF publishers)

```
$ ros2 topic info /tf -v
Type: tf2_msgs/msg/TFMessage

Publisher count: 2

Node name: robomaster
Node namespace: /
Topic type: tf2_msgs/msg/TFMessage
Topic type hash: RIHS01_e369d0f05a23ae52508854b66f6aa0437f3449d652e8cbf22d5abe85d020f087
Endpoint type: PUBLISHER
GID: 01.0f.f0.2e.5d.27.b9.c5.00.00.00.00.00.00.15.03
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

Node name: robot_state_publisher
Node namespace: /
Topic type: tf2_msgs/msg/TFMessage
Topic type hash: RIHS01_e369d0f05a23ae52508854b66f6aa0437f3449d652e8cbf22d5abe85d020f087
Endpoint type: PUBLISHER
GID: 01.0f.f0.2e.5e.27.68.dd.00.00.00.00.00.00.14.03
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
... 89 lines omitted ...
```

Figure 28: Publishers of `/tf` (`logs/ap_tf_info.txt`).

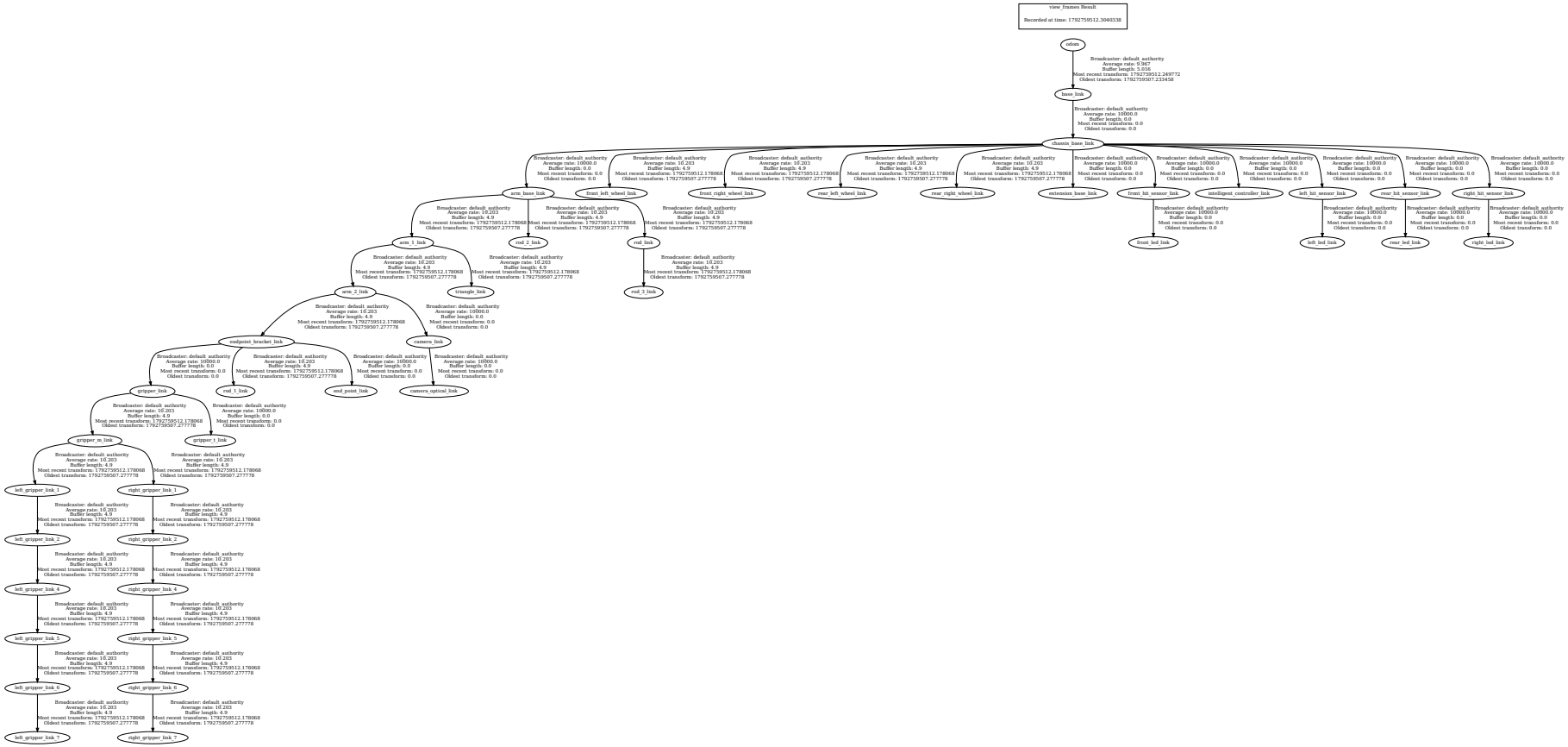


Figure 29: The TF tree generated by `tf2_tools view_frames` (vector PDF; zoom in for the broadcaster, rate and buffer data on every edge).

Deliverables checklist

Item	Status	Where
System diagnostics (kernel, Python, ROS 2, NumPy)	done	Figures 2, 3, 4
NumPy ABI analysis and isolation method	done, reproduced	Figure 5, Figure 4
Wired and AP latency plots (telemetry Δt, ping RTT)	done	Section 3.4
Reconstructed odometry trajectory (X vs Y)	done; error needs a tape measurement	Figure 24
Table of topology comparison	done	Problem 4(i)
Derivation of $\mathbf{J}_{IK}$, $\mathbf{J}_{FK}$	done, verified	Problem 4(ii)
TF tree and explanation	done	Problem 4(iii)
`ep_teleop_sdk.py`, `rm_setup.sh`, raw CSV logs	done	repository, Appendices A and B
ROS 2 run with lateral strafing and measured end position	strafing run done over USB; **[TODO: tape-measured end position missing]**	Problem 3(ii), 3(iv)
Video (a) SDK teleoperation, (b) ROS 2 teleoperation over AP	to be added	**[ADD VIDEO LINK HERE]** and **[ADD VIDEO LINK HERE]** (edit the boxes at the top of the source)

Discussion and limitations

- **Each configuration was measured once.** The wired and AP runs are single 30 s logs and one 100-packet ping per link; the ranges quoted (for example the 152 ms maximum) describe those runs, and a Wi-Fi link in a different environment would give different tails.
- **Time resolution.** The telemetry is logged at 10 Hz, so response delays have a resolution of 0.1 s, and means of 0.16 s and 0.19 s should not be over-interpreted; the 0.30 s versus 0.50 s worst cases are the more reliable difference.
- **Wheel order.** The assumption that the SDK's ESC order is FR, FL, RL, RR was verified from the data (Table 2).
- **Nominal dimensions.** The wheel radius and chassis half-dimensions used in the numerical examples are the nominal EP values, not measured on this robot.
- **Open items.** A ROS 2 run with lateral strafing, the tape-measured end position for the odometry error, and the two demonstration videos.

Listing of `ep_teleop_sdk.py`

```
#!/usr/bin/env python3
"""Standalone RoboMaster EP keyboard teleoperation + 10 Hz telemetry logger.

Uses only the native `robomaster` Python SDK (no ROS 2 imports).

Keys (curses, non-blocking):
    W / S   forward / backward         (vx, m/s)
    A / D   strafe left / right        (vy, m/s)
    Q / E   yaw left (CCW) / right (CW) (wz, deg/s)
    SPACE   immediate stop (zero all velocities)
    X / ESC quit

Terminals only deliver key *presses* (auto-repeat), never releases, so a key
counts as "held" while repeats keep arriving.  When they stop for
HOLD_TIMEOUT seconds the key is treated as released and the chassis stops.

SDK sign convention: x forward, y RIGHT, z CLOCKWISE (deg/s).

Usage:
    python ep_teleop_sdk.py --conn rndis --duration 30 --out logs/rndis_run.csv
    python ep_teleop_sdk.py --conn ap    --duration 30 --out logs/ap_run.csv
"""
import argparse
```

```
import csv
import curses
import os
import threading
import time

from robomaster import robot

LOG_HZ = 10.0           # telemetry log rate
CMD_HZ = 20.0           # command dispatch rate
FIRST_PRESS_HOLD = 0.55  # covers the OS typematic delay before auto-repeat
REPEAT_HOLD = 0.15       # hold window once auto-repeat is running

# key -> (vx, vy, wz) direction
KEYMAP = {
    ord('w'): (1, 0, 0), ord('s'): (-1, 0, 0),
    ord('a'): (0, -1, 0), ord('d'): (0, 1, 0),
    ord('q'): (0, 0, -1), ord('e'): (0, 0, 1),
}

CSV_HEADER = [
    't_rel_s', 't_unix', 'batt_pct', 'yaw_deg', 'pitch_deg', 'roll_deg',
    # SDK ESC order: wheel1=front-right, wheel2=front-left, wheel3=rear-left, wheel4=rear-right
    'w_fr_rpm', 'w_fl_rpm', 'w_rl_rpm', 'w_rr_rpm',
    't_esc_rx_unix',  # arrival time of the newest ESC packet (network jitter)
    'cmd_vx', 'cmd_vy', 'cmd_wz',
]


class Telemetry:
    """Thread-safe latest-value store filled by the SDK subscription callbacks."""

    def __init__(self):
        self.lock = threading.Lock()
        self.batt = float('nan')
        self.att = (float('nan'),) * 3
        self.esc = (float('nan'),) * 4
        self.esc_rx = float('nan')

    def on_batt(self, percent):
        with self.lock:
            self.batt = percent

    def on_att(self, info):
        with self.lock:
            self.att = tuple(info[:3])

    def on_esc(self, info):
        with self.lock:
            self.esc = tuple(info[0])
            self.esc_rx = time.time()

    def snapshot(self):
        with self.lock:
            return self.batt, self.att, self.esc, self.esc_rx


def active_command(deadlines, vx_max, vy_max, wz_max):
    """Sum the direction of every key whose hold window has not expired."""
    now = time.monotonic()
    vx = vy = wz = 0.0
    for key, deadline in list(deadlines.items()):
        if deadline < now:
            del deadlines[key]
            continue
        dx, dy, dz = KEYMAP[key]
        vx, vy, wz = vx + dx * vx_max, vy + dy * vy_max, wz + dz * wz_max
    return vx, vy, wz


def run(stdscr, args):
    curses.curs_set(0)
    stdscr.nodelay(True)
    stdscr.keypad(True)

    ep = robot.Robot()
    ep.initialize(conn_type=args.conn)
    chassis, battery = ep.chassis, ep.battery
    tel = Telemetry()
    battery.sub_battery_info(freq=10, callback=tel.on_batt)
    chassis.sub_attitude(freq=10, callback=tel.on_att)
    chassis.sub_esc(freq=10, callback=tel.on_esc)

    os.makedirs(os.path.dirname(os.path.abspath(args.out)), exist_ok=True)
    deadlines = {}
    t0 = time.monotonic()
    next_log = t0 + 1.0 / LOG_HZ
    next_cmd = t0
    cmd = (0.0, 0.0, 0.0)
    rows = 0
    status = 'running'

    with open(args.out, 'w', newline='') as fh:
        writer = csv.writer(fh)
        writer.writerow(CSV_HEADER)
        try:
            while True:
                now = time.monotonic()
                if args.duration and now - t0 >= args.duration:
                    status = 'duration reached'
                    break
```

```
125
126                 # ---- keyboard ----
127                 key = stdscr.getch()
128                 while key != -1:  # drain everything queued since last loop
129                     k = key if key > 255 else (ord(chr(key).lower()) if 0 <= key < 256 else key)
130                     if k == ord(' '):
131                         deadlines.clear()
132                         cmd = (0.0, 0.0, 0.0)
133                         chassis.drive_speed(x=0, y=0, z=0)  # failsafe, immediate
134                     elif k in (ord('x'), 27):
135                         status = 'quit by user'
136                         raise StopIteration
137                     elif k in KEYMAP:
138                         hold = REPEAT_HOLD if k in deadlines else FIRST_PRESS_HOLD
139                         deadlines[k] = now + hold
140                     key = stdscr.getch()
141
142                 # ---- command dispatch ----
143                 if now >= next_cmd:
144                     cmd = active_command(deadlines, args.vx, args.vy, args.wz)
145                     chassis.drive_speed(x=cmd[0], y=cmd[1], z=cmd[2])
146                     next_cmd += 1.0 / CMD_HZ
147
148                 # ---- 10 Hz telemetry log ----
149                 if now >= next_log:
150                     batt, att, esc, esc_rx = tel.snapshot()
151                     writer.writerow([f'{now - t0:.4f}', f'{time.time():.4f}', batt,
152                                      *att, *esc, f'{esc_rx:.4f}', *cmd])
153                     rows += 1
154                     next_log += 1.0 / LOG_HZ
155                     stdscr.erase()
156                     stdscr.addstr(0, 0, f'RoboMaster EP teleop [{args.conn}]  t={now - t0:6.1f}s  rows={rows}')
157                     stdscr.addstr(1, 0, 'W/S fwd/back  A/D strafe  Q/E yaw  SPACE stop  X quit')
158                     stdscr.addstr(3, 0, f'cmd  vx={cmd[0]:+.2f} m/s  vy={cmd[1]:+.2f} m/s  wz={cmd[2]:+.1f} deg/s')
159                     stdscr.addstr(4, 0, f'batt {batt:5.1f} %')
160                     stdscr.addstr(5, 0, 'yaw/pitch/roll  %7.2f %7.2f %7.2f deg' % att)
161                     stdscr.addstr(6, 0, 'wheels FR/FL/RL/RR  %6.0f %6.0f %6.0f %6.0f rpm' % esc)
162                     stdscr.refresh()
163
164                 time.sleep(0.005)
165         except (StopIteration, KeyboardInterrupt):
166             pass
167         finally:
168             # failsafe: always leave the chassis stopped and subscriptions closed
169             for _ in range(3):
170                 chassis.drive_speed(x=0, y=0, z=0)
171                 time.sleep(0.02)
172             chassis.unsub_esc()
173             chassis.unsub_attitude()
174             battery.unsub_battery_info()
175             ep.close()
176     return status, rows
177
178
179 def main():
180     ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
181     ap.add_argument('--conn', choices=['rndis', 'ap'], required=True,
182                     help='rndis = wired USB (robot 192.168.42.2); ap = direct Wi-Fi hotspot')
183     ap.add_argument('--out', default='logs/telemetry.csv', help='CSV output path')
184     ap.add_argument('--duration', type=float, default=0.0, help='auto-stop after N seconds (0 = run until X)')
185     ap.add_argument('--vx', type=float, default=0.3, help='forward speed magnitude, m/s')
186     ap.add_argument('--vy', type=float, default=0.3, help='strafe speed magnitude, m/s')
187     ap.add_argument('--wz', type=float, default=45.0, help='yaw rate magnitude, deg/s')
188     args = ap.parse_args()
189     status, rows = curses.wrapper(run, args)
190     print(f'{status}: wrote {rows} telemetry rows to {args.out}')
191
192
193 if __name__ == '__main__':
194     main()
```

Listing of `rm_setup.sh`

```
1  #!/usr/bin/env bash
2  # Workspace environment loader for the RoboMaster EP assignment.
3  # Usage:  source ~/rm_setup.sh
4  #
5  # Order matters: ROS 2 and the workspace overlay first (they set PYTHONPATH),
6  # then the venv, then pin the interpreter's search path so ROS binary bindings
7  # (rclpy, cv_bridge) are found but user-site packages cannot shadow them.
8
9  source /opt/ros/jazzy/setup.bash
10 [ -f "$HOME/ros2_ws/install/setup.bash" ] && source "$HOME/ros2_ws/install/setup.bash"
11
12 # venv was created with --system-site-packages, so it sees /opt/ros/jazzy/... via PYTHONPATH
13 source "$HOME/rm_venv/bin/activate"
14
```

```
# Make sure the venv's site-packages (numpy<2, robomaster SDK) win over ROS's
# dist-packages, and that ~/.local packages (possibly numpy 2.x) are ignored.
VENV_SITE="$(python -c 'import sysconfig; print(sysconfig.get_paths()["purelib"])')"
export PYTHONPATH="$VENV_SITE:/opt/ros/jazzy/lib/python3.12/site-packages:${PYTHONPATH}"
export PYTHONNOUSERSITE=1

export ROS_DOMAIN_ID="${ROS_DOMAIN_ID:-0}"
```

Repository layout and commands

```
Assignment 1/
  ep_teleop_sdk.py            keyboard teleoperation + 10 Hz telemetry (SDK only)
  rm_setup.sh                 environment loader
  run_ap_all.sh               scripted AP session
  scripts/                    ping_bench.sh, analyze_telemetry.py, plot_odom.py,
                              command_lag.py, kinematics_check.py, make_report_figs.py, term_shot.py
  logs/                       csv logs, pings, bag_ap/, captures/
  figures/                    plots, term/ (terminal captures), frames.pdf
  report/report.tex           this report


python ep_teleop_sdk.py --conn rndis --duration 30 --out logs/rndis_run.csv
python ep_teleop_sdk.py --conn ap    --duration 30 --out logs/ap_run.csv
python scripts/analyze_telemetry.py --rndis-csv logs/rndis_run.csv --ap-csv logs/ap_run.csv \
       --rndis-ping logs/ping_rndis.txt --ap-ping logs/ping_ap.txt --out figures
python scripts/make_report_figs.py        # all figures of this report (needs the ROS environment)
```